

Magneto: Load-Balanced Key-Value Service for Write-Intensive Workloads

Yuanhang Gao , Yingwen Chen, Xiangrui Yang , Huan Zhou , Shihua Tang, and Ming Xu

Abstract—High-performance key-value (KV) storage is critical for the cloud, providing KV services to various cloud applications. A key challenge for KV services is that workloads of cloud applications are often write-intensive and exhibit highly-skewed characteristics, which result in load imbalance among storage servers thus lowering system performance. To address this problem, prior works set up an in-switch write-back caching mechanism, which adopts a centralized controller to balance writes. However, the limited bandwidth between the switches and the controller is a severe bottleneck for achieving high performance. In this paper, we present Magneto, a novel key-value service architecture for the cloud. At the core of Magneto is a delayed-write mechanism in the switch data plane to absorb frequent write queries for hot items. It effectively balances the load under write-intensive workloads without involving the controller. Magneto also designs a reliability mechanism to ensure system reliability during switch state transitions and failures. We implement a prototype using an FPGA-integrated switch, which has high packet-processing performance and contains enough memory to provide both the in-switch cache and the write buffer. Extensive evaluation shows that Magneto can achieve 8.4x system throughput gains compared to baseline systems when handling a skewed workload consisting of 70% reads and 30% writes. Moreover, it can reduce the load on back-end servers up to 56% in total.

Index Terms—Key-value stores, in-network processing, programmable hardware, distributed system.

I. INTRODUCTION

KEY-VALUE (KV) storage plays a vital role in the cloud, providing KV services to various cloud applications such as web search [1], social networks [2], and large-scale machine learning [3]. Recent advancements in large language models (LLMs) have pushed the demand higher, especially for write-intensive workloads [4] (for example, write ratio $> 10\%$). These workloads require frequent updates to key-value pairs, increasing pressure on key-value store architectures to maintain high performance and reliability. For example, UP2X, a distributed key-value store in Meta, supports AI / ML services by storing dynamic profile data (e.g., user counters) that are frequently updated. More than 92% of its 100+ million daily queries are

read-modify-write operations [4], highlighting the prevalence of write-intensive workloads in real-world applications. More than 35% of all Twemcache clusters have a high proportion of writes, and more than 20% experience more writes than reads. Furthermore, these write-intensive workloads exhibit skewness, with 25% of frequently accessed (or “hot”) records dominating the write workloads in Twitter’s Twemcache clusters [5], [6]. Similar locality of write queries is also observed in Meta [4].

In distributed KV services, clients issue queries that are later delivered to different back-end servers where the requested contents live. Particular servers may receive frequent queries and be overloaded under highly skewed workloads. Some queries may suffer from longer queueing delays or dropouts due to an imbalance load among storage servers. To cope with this problem, previous work [7] deploys cache mechanisms on programmable switches to absorb read queries from back-end servers on a rack. In such cases, read queries for hot items can be served directly on the switch with lower latency and higher throughput. However, when receiving write queries, the switch needs to invalidate the corresponding key-value pairs in the cache and forward them to back-end servers. This results in severe in-switch cache misses in the case of write-intensive workloads, thereby failing to balance load under write-intensive workloads. This issue also exists in other caching mechanisms [8], [9]. Although FarReach [6] tried to address this issue with programmable switches, it needs to involve a controller to achieve system reliability and data consistency. Specifically, the centralized controller needs to maintain key-value pair version control and snapshots, which poses performance challenges due to the limited bandwidth between the controller and switches. In summary, existing approaches to handling write-intensive workloads either suffer from cache breakdown or rely heavily on centralized controllers.

To keep the performance aligned with the modern link speed in large-scale networks, it is important to address the problem without involving the controller. For this purpose, we use the skewness of write queries observed in key value workloads by proposing an in-switch write buffer (referred to as the in-switch stash). The stash mechanism is designed to absorb frequent write queries and reduce the memory-access load on the back-end servers. It also enables read queries to retrieve the latest version of key-value pairs stored in the stash, thus suppressing the increase of switch misses caused by write queries. However, handling writing queries purely on the switch data plane is non-trivial due to two challenges. First, setting up an in-switch

Received 16 July 2024; revised 22 May 2025; accepted 26 June 2025. Date of publication 24 July 2025; date of current version 9 October 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62372462, in part by the Natural Science Foundation of Hunan Province under Grant 2023JJ40682, and in part by the Youth Innovation Fund of the National University of Defense Technology under Grant ZK2023-13. (Corresponding author: Xiangrui Yang.)

The authors are with the College of Computer Science and Technology, National University of Defense Technology, Changsha 410003, China (e-mail: ywch@nudt.edu.cn; yangxiangrui11@nudt.edu.cn).

Digital Object Identifier 10.1109/TSC.2025.3592425

stash to process write queries instead of directly forwarding them to servers raises data consistency issues between switches and back-end servers. This necessitates an efficient synchronization mechanism. Second, key-value stores, as a critical component of many distributed applications, must ensure system reliability, even in the case when the switch experiences state transition or failure. Therefore, a robust reliability mechanism is needed under such circumstances.

This paper introduces Magneto, a lightweight, fast, and reliable architecture designed to optimize the system's overall performance under write-intensive workloads in distributed key-value stores. Magneto incorporates an in-switch stash mechanism to buffer write queries in the network, absorbing frequent write queries for hot items instead of forwarding them directly to the back-end servers. Specifically, the stash holds write queries in the switch's data plane buffer for a short period, allowing the switch to respond to some cache-missed read queries and merge multiple writes into one. This approach mitigates the load on the back-end servers. And cold write queries are sent to the back-end servers after a short buffering period. To the best of our knowledge, this is the first work to propose an in-network write buffer for handling write-intensive key-value workloads. Moreover, Magneto achieves a reliability mechanism to ensure system reliability even during switch state transition or failure. Magneto is lightweight, compatible with existing network protocols and infrastructure, and can be easily integrated with existing key-value databases [10], [11]. It targets modern rack-scale storage clusters that contain thousands of cores per rack.

The emergence of LLMs is beginning to profoundly impact our work and daily routines [12], [13]. In our view, the concepts behind Magneto can be applied to LLMs as well. For example, in distributed training, parameter servers must handle frequent gradient updates (e.g., via AllReduce operations), where high write frequency can become a bottleneck. The concept of delayed writes can be applied to the NIC of parameter servers, where multiple gradient updates are merged into a single submission via deferred commits, thereby reducing communication overhead.

In summary, this paper makes the following contributions:

- We propose Magneto, a lightweight, fast, and reliable in-network key-value store architecture. Magneto effectively balances the load among back-end servers under write-intensive workloads using the switch data plane.
- We designed a delayed write mechanism that buffers and merges write queries in the in-switch stash to reduce the load sent to servers and improve switch misses.
- We designed a reliability mechanism where the client backs up the unacknowledged write queries until receiving a response. In case of switch failure or state transitions, the lost data can be recovered from the backups distributedly stored by the clients.
- We evaluated Magneto and found that it can achieve a 8.4x increase in system throughput compared to baseline systems when handling a skewed workload consisting of 70% reads and 30% writes. Additionally, it is able to reduce the total load received by the storage servers to only 56% of the original load.

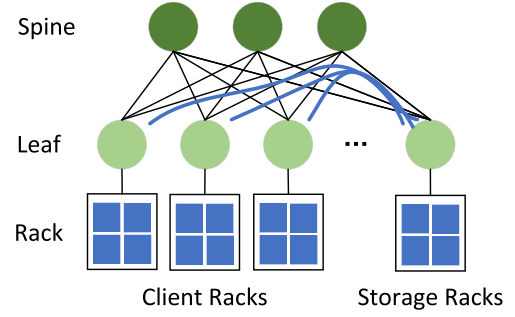


Fig. 1. Datacenter network architecture.

II. BACKGROUND AND MOTIVATION

A. Key-Value Stores in Data Center

A key-value store is a data storage model that organizes data into key-value pairs, where each key uniquely identifies a corresponding value [1], [14], [15]. This infrastructure is employed in various computing tasks requiring fast data access, insertion, and deletion operations, including web service, session storage, and Machine Learning. Key-value stores serve as fundamental components in modern computing systems, enabling rapid and efficient data storage and retrieval operations across diverse applications.

Modern data centers employ distributed Clos/Spine-Leaf topologies, shown in Fig. 1, where servers connect redundantly to multiple leaf switches, creating a many-to-many communication pattern that ensures bisection bandwidth and fault tolerance. This typical networks in mainstream data centers decouple computation and storage resources into different racks for scalability, lower cost, and higher resource utilization. In this setup, the Spine switches act as the core switch connecting to various Leaf switches, which are also known as ToR (Top-of-Rack) switches. All client queries related to key-value traffic are routed through the Spine switches and subsequently through the ToR switches to access the storage servers. After processing, the responses back to the respective client situated in the Client Racks, which may consist of individual virtual machines or containers.

B. Motivation

In distributed key-value stores, the ideal case is to evenly distribute key-value store workloads among storage servers and keep hot items on the switch to maximize the overall performance. Specifically, if each storage node has a throughput of T , a key-value store with N nodes can achieve an overall throughput of $N \cdot T$. As the load allocated to each node does not exceed T , the query latency is bounded and also allows service providers to scale out a storage system as needed easily. However, the real-world workloads follow the Zipf distribution [16]:

$$f(x) = \frac{1}{x^\alpha \sum_{i=0}^{\infty} \frac{1}{i^\alpha}} \quad x = 0, 1, 2, \dots$$

The description of this distribution is as follows: x represents the key, and $f(x)$ represents the number of times x is queried. When $\alpha > 1$, it indicates that the workloads are highly skewed. [5] shows that most of the α values are in the range from 1 to 2.5, indicating the workloads are highly skewed in Twitter. Hot items

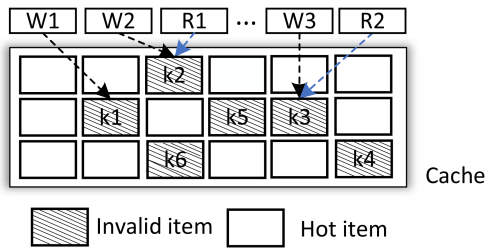


Fig. 2. Widespread cache misses under writes.

account for most queries, causing load imbalance on storage servers. Specifically, overloaded nodes experience higher query volumes and longer queuing delays, while other nodes remain underutilized. Both of these factors prevent achieving the ideal overall throughput. The root cause of this load imbalance lies in highly skewed workloads. There are two natural methods: 1. distributing hot items across storage servers to reallocate workloads equally; 2. reducing the load of hot queries before sent to storage servers. The first method requires dynamically allocating hot items to different servers, which consumes CPU resources compared to in-network processing and also increases RTT (Round-Trip Time). We focus on reducing the load of hot queries before they reach servers, like the second method.

In NoSQL databases [10], [11], operations like set, add, replace, append, cas, incr, decr, and prepend all involve modifications to key-value pairs [5]. Hence, they can be categorized as “write” operations, making queries of key-value store workloads to be classified into reads, writes, and deletes. The proportion of write-intensive workloads is gradually rising and also exhibits skewness in clusters. NetCache [7] leverages the skewness of read-intensive workloads and sets up a cache in the ToR switches to absorb hot reads. However, when receiving writes, it invalidates relevant items in the cache and then directly forwards them to storage servers. This leads to widespread in-switch cache misses when handling write-intensive workloads, as demonstrated in Fig. 2. Because read and write queries follow the same distribution, the requested key-value pairs overlap significantly [4], [7]. Thereby it fails to balance the load among servers. Then FarReach [6] designs a more complex in-switch cache that involves cooperation between the controller and the data plane to address this issue. However, the limited bandwidth between the switches and controller poses a bottleneck for efficiently achieving data version control and snapshots.

C. In-Switch Stash and FPGA Integration

Setting up an in-switch Stash: Skewed writes greatly increase the imbalanced load on storage servers. However, if we could set up an in-switch buffer to temporarily store write requests and delay committing them to the servers, it can effectively reduce the number of queries sent to the servers. The reasons are:

- 1) Writes that invalidate read cache records are temporarily stored in the buffer, allowing cache-missed read queries to still retrieve data from the buffer.
- 2) It merges multiple write requests for the same key, as this operations have no impact on the final result.

So we set up an in-switch stash to buffer writes for a while before forwarding them to servers. In theory, by setting an appropriately sized stash, most of the write queries sent to the servers can be absorbed. If combined with a cache, then no node would experience more than T load with high probability, regardless of the query distribution.

Leveraging FPGA-integrated switch for in-network processing: Deploying a dedicated load-balancer server in front of storage servers can also help mitigate the impact of skewed load. But servers generally have lower I/O performance compared to switches, which could be a bottleneck for overall performance. For example, one switch has equivalent I/O performance of 20 x86 servers [17]. Therefore, utilizing in-network caching with switches, especially the ToR switch, is a more suitable option, as it is located in the critical path, incurring no unnecessary latency.

Programmable switch, such as Barefoot Tofino [18], can process several billion packets per second and empower users to re-program the packet-processing pipeline using P4 [19] without changing hardware. Its pipeline consists of multiple Match-Action stages, where the size of the key for matching is fixed, and the stateful memory is distributed among stages with no sharing between them [20], [21], [22]. It owns several tens of megabytes of stateful memory, which is abstracted as register arrays in each pipeline stage. To accommodate variable-length values in in-memory key-value stores, register arrays from multiple stages are concatenated to construct a large value, with the premise of meeting strict resource and timing requirements. The limited stateful memory [23] and strict programming rules make it challenging for programmable switches to simultaneously implement cache and stash, even if the cache only holds the hottest $O(N \log N)$ items [24]. Therefore, we implement a prototype using an FPGA-integrated switch by attaching the FPGA to the switch to demonstrate the efficiency of our designed methods. This setup leverages both the high I/O performance of the switch and the ample memory resources of the FPGA. Additionally, our core hardware primitives can be migrated to future programmable packet processing pipelines to achieve higher performance.

III. MAGNETO DESIGN

In this section, we present a novel in-network caching architecture for key-value store named Magneto. As an overview, Magneto leverages programmable hardware to balance load under highly skewed writes, preventing storage servers from being overloaded. It focuses on resolving load imbalances among storage servers under write-intensive workloads and targets the following goals:

- *Balancing load under write-intensive workloads:* The design of Magneto needs to absorb skewed writes targeting a small proportion of hot key-value pairs in the switch and reduce switch misses under write-intensive workloads.
- *Ensuring system reliability:* Its design needs to ensure system reliability even under extreme scenarios where switch failure or state transitions.
- *Demonstrating data consistency:* Its design needs to maintain data consistency between switches and servers by

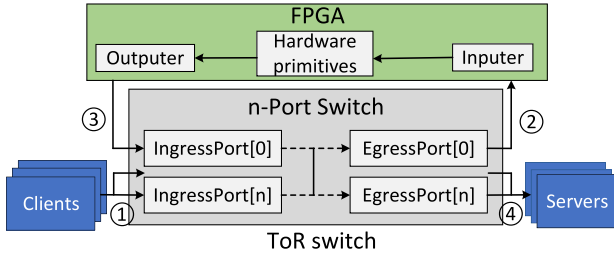


Fig. 3. Megneto overview.

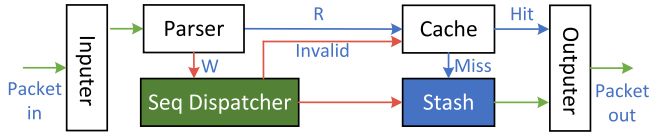


Fig. 4. Structures of appended hardware primitives. The green arrow represents mixed read and write queries. After parsing, read queries (blue arrow) are sent to Cache, and write queries (red arrow) are sent to Seq dispatcher.

synchronizing newer data from switch to the back-end servers.

The architecture of Megneto, as shown in Fig. 3. In Megneto, the ToR switch is composed of a switch and an FPGA. The switch directs the received key-value traffic to the FPGA for in-network processing and then forwards it after processing is complete. Specifically, the ToR switch receives traffic from clients through IngressPort[1] to IngressPort[n], as shown in step ① of the diagram. If it is key-value traffic, it is forwarded to EgressPort[0] and sent to the FPGA, as shown in step ②. After processing by the FPGA, the key-value data re-enters the switch via IngressPort[0], as shown in step ③. Finally, it is forwarded out through EgressPort[1] to EgressPort[n], as shown in step ④. The traffic traverses paths in the FPGA as illustrated in Fig. 4. Queries originate from the client side and must traverse through ToR switches to reach the storage servers. Megneto deploys a stash module in the ToR switch to absorb frequent writes, protecting servers from overload, which resembles delayed writes. Additionally, it initiates data synchronization between switches and servers actively to ensure data consistency. Clients are required to assign a sequence number (seq) to each write query and back up the data, enabling recovery from backups in case of switch failures.

We follow the network protocol design of prior work [7], which is based on TCP/UDP and appending OP, Seq, Key, and Values as payloads. Clients use either TCP or UDP based on their specific needs. There are 7 types of packets, where OP=4 and OP=5 packets support Cache and Stash functionalities, respectively, while the remaining correspond to different types of key-value stores queries. The three parties - clients, switches, and servers - will implement all mechanisms of Megneto through interactions using these 7 types of packets, without involving a limited-bandwidth controller. We will explain these mechanisms in detail in the following sections.

A. Delayed Writes in Megneto

Essence of load imbalance under skewed workloads: Before introducing delayed write, we need to understand the essence

of load imbalance among storage servers under skewed loads. Prior studies have shown that 10% of items account for 60-90% of queries in the Memcached deployment at Meta[4]. We refer to these 10% of items as “hot items”. The majority of query traffic floods toward storage servers that hold hot items, resulting in overload, reduced performance, and longer latency. As the read/write ratio decreases in Memcached clusters, the proportion of write queries increases significantly. Setting up a cache in switches hardly prevents load imbalance under write-intensive loads, necessitating a new module to support skewed writes.

Design of in-switch stash: We propose a novel mechanism that adds a stash module in the switches to absorb frequent writes for “hot items”, thereby relieving load on the storage servers. This mechanism, referred to as delayed write, is described in Fig. 5. When a client issues a write query (e.g., for a ‘red’ key), Megneto stores this write in the stash and initializes a counter (i.e., recording the count of key X writes) to 1, as shown in Fig. 5(a). In the next step, four write queries are entered consecutively, with three keys being red and one being orange. As a result, the counter for the red key is incremented to 4, while the orange key remains at 1. This demonstrates how the stash buffers writes and holds the latest version of key-value data. The stash is required to push the newest data to the storage servers for two reasons: first, as writes with different keys continue to be placed into the stash, it is expected to become full and lose its ability to accommodate new writes; second, the stash cannot replace the role of servers. Therefore, synchronization is triggered when the occupancy rate of the stash exceeds the predetermined threshold k , depicted by the red dashed line in Fig. 5. The stash generates SYN packets and sends them to servers during synchronization.

In step 3, the stash receives some writes. For keys that did not previously exist, they will be buffered, and the counter is initialized to 1. For keys that already exist in the stash, the corresponding value is updated, and the counter is incremented by 1. After step 3, the occupancy rate exceeds k , triggering synchronization. During synchronization, three actions occur: (1) batch packaging and sending key-value pairs to servers via SYN packets; (2) resetting all counters to 1; and (3) pausing occupancy rate checks until a REPLY packet arrives.

During the synchronization period, the stash can still receive new writes, as shown in step 5 of Fig. 5, and continues to function normally. In Fig. 5(c), deep orange represents a new write received during synchronization. In step 6, the switch receives a REPLY packet from the server, indicating that synchronization is complete. For the purple key and all key-value pairs preceding it, the corresponding counters are decremented by 1. If the counter value for a certain key becomes 0, that key will be removed from the stash, similar to the yellow key in the illustration. After these six steps, only the red and orange keys remain in the stash, indicating that they are either hot items or new additions.

Key details to note: Since the stash always buffers write queries before forwarding them to servers, all key-value pairs in the stash are newer than their copies in the servers when synchronization is triggered. To ensure data consistency, we batch the data into SYN packets before sending them to servers to reduce the time consumed by synchronization. Although frequently updated pairs may have large counter values, synchronization occurs only once. Therefore, setting the counter values of all

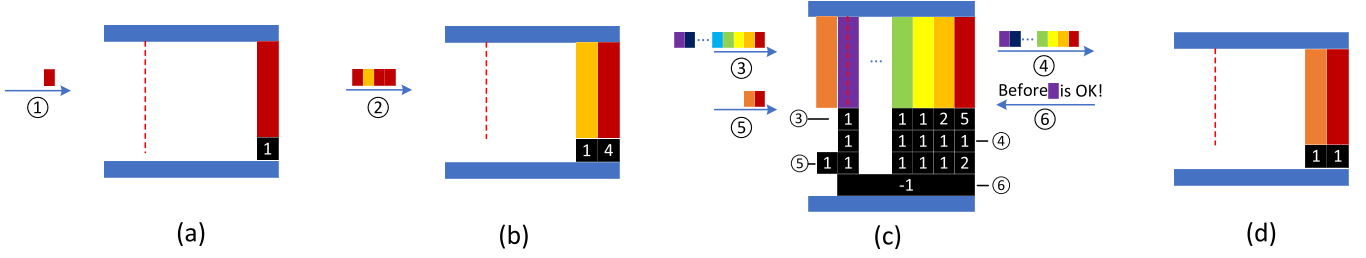


Fig. 5. Delayed writes in Magneto. The four sub-figures sequentially depict the changes in the stash under incoming write queries. Different colors represent different keys, and 1–6 denote the changes over time. The red dashed line indicates the condition triggering the synchronization of stash.

existing pairs to 1 can prevent syncing the same pair multiple times, thus avoiding wasting bandwidth between switches and servers.

Sync is triggered when the occupancy of the stash exceeds k . However, having only this condition could result in multiple syncs being triggered, even if the first sync has not been completed. Furthermore, subsequent REPLY packets from switches may erroneously decrement the counter values of hot items, leading to their removal from the stash and reducing its efficiency. To address this issue, we specify that no new sync should be triggered until the ongoing sync is complete, meaning until a REPLY packet is received. This ensures that synchronization processes are properly managed and completed before initiating new ones, preserving the integrity and efficiency of the stash mechanism. In low-traffic nighttime scenarios, write requests may remain in the Stash for an extended period, increasing response latency and potentially violating Service Level Objectives (SLOs). To address this, we introduce a maximum synchronization interval T_s . When the time since the last synchronization exceeds T_s , a new synchronization is triggered regardless of the current Stash occupancy. This mechanism ensures a consistent user experience even during off-peak hours.

Method rationale: delayed write: First, setting up an in-switch stash can merge multiple writes into a single write without impacting the final result. Second, ensuring that only frequently written key-value pairs remain in the stash for extended periods prioritizes caching for items that are heavily accessed or updated. This ensures that the stash efficiently serves its purpose of absorbing frequent write queries for hot items and acts as another “cache” for read queries to reduce massive switch misses under write-intensive workloads, thereby effectively balancing the load on storage servers and improving overall system performance.

B. System Reliability in Magneto

Since the in-switch stash keeps the latest records under our delayed write design, as mentioned before, we need to protect against data loss during switch state transitions or failures. According to preliminary estimates, the number of the latest records buffered in the stash is about 10,000 per switch. A previous study on Twitter clusters showed that the average size of keys is 32 bytes, while the average size of values is 128 bytes, totaling 160 bytes per key-value pair [5]. Using the controller to back up newer records for service recovery in case of switch failure poses

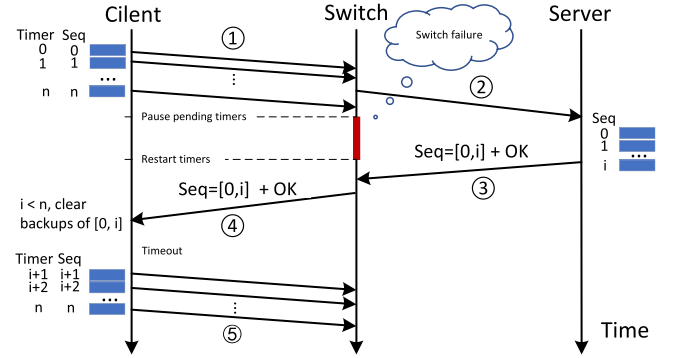
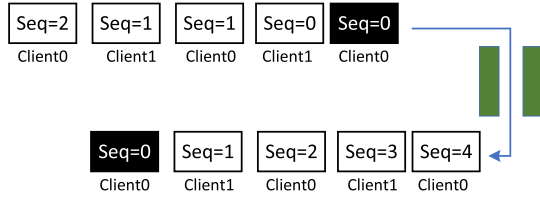


Fig. 6. Reliability mechanisms in magneto. Synchronization needs a duration of time. If the switch fails during synchronization, resulting a loss of $[i+1, n]$ data. Upon receiving the response from the server, client clears the backup of data indices $[0, i]$. Subsequently, the following events occur on the client side: 1. Timer $i+1$ timeout; 2. Resend $[i+1, n]$ data; 3. Reset rest Timers.

a challenge due to the limited bandwidth between the switch and controller. Additionally, because records are continuously updated, there is a need to store 1.6 MB of data within a very short time window, further exacerbating the challenge. We aim to propose a new mechanism that avoids using the controller to back up data, thus fundamentally avoiding potential bottlenecks. Moreover, it should not compromise the overall I/O performance of the whole system.

The reliability mechanism: We propose a three-party collaboration mechanism for Magneto to ensure system reliability even under switch failures, without involving the controller. The three parties involved are clients, switches, and servers, with a focus on the design of the client side, as shown in Fig. 6. Specifically, we require the client to perform the following actions before sending a write query: 1. Back up the key-value data locally; 2. Add an incremental sequence number to the packet; 3. Set a timeout timer. After that, clients wait for WRITE_REPLY packets from servers.

When write queries enter the switch, they will be buffered in the stash before synchronization. During this period, subsequent writes with the same key may arrive and overwrite previous ones. As mentioned earlier, later arriving writes have a larger sequence number, implying that overwritten writes will not appear in the SYN packets. After receiving all the SYN packets, which contain multiple writes with discontinuous sequence numbers, the server processes them. The server then constructs a WRITE_REPLY packet when processing is complete. In this packet, two fields—Seq and “OK”—follow the TCP/UDP fields, and this



(a) The green part functions as a Seq dispatcher, assigning incoming packets a global SEQ.

	Client	Max_seq	Local_seq
Client0	(192.0.2.1:8888)	4	2
Client1	(104.25.6.28:9999)	3	1
	...	...	...
ClientN	(203.0.13.10:9999)	415	24

(b) Record the mapping between global seq and local seq for each client.

Fig. 7. Local seq and global seq.

Seq is equal to the largest sequence number of writes processed by the server during this synchronization. Similar to Go-back-to-N, the Seq field in the WRITE_REPLY packet indicates that all writes with sequence numbers preceding this Seq have been processed. The switch decrements the counter values of all records in the stash that have a smaller sequence number than Seq when receiving WRITE_REPLY packets and forwards them to clients. Upon receiving the WRITE_REPLY packet, clients delete backups and timers associated with sequence numbers below the acknowledged Seq.

In fault-free scenarios, the additional overhead of backing up writes in clients and setting timeout timers seems unnecessary because clients only need to back up their own writes. However, fault-free scenarios are nonexistent. Data stored in the switch's in-memory cache will be lost and cannot be recovered in cases of switch failure or power outage. In Magneto, if clients detect a switch failure through the heartbeat mechanism, they first pause all timers for backups that have not received server acknowledgments and stop sending new queries until the switch resumes normal state. Once the switch recovers, all paused timers are restarted, and backups are sent in the order of timeout until the clients receive acknowledgments from the servers. During the period of restoring the original state, no new queries are sent.

Local seq and global seq: In key-value stores, the Seq value added by each client before sending a write is incremented based on the number of writes it has already sent. Consequently, different clients may inevitably send writes with the same Seq at different times, leading to duplicate and non-sequentially increasing Seq values in the writes received by the switch. However, the premise assumption for the operation of the stash mechanism mentioned above is that subsequent writes have larger Seq values. To address this issue, it is necessary to convert the local Seq added by each client into a globally unique and incrementally increasing Seq. Clients cannot obtain a globally unique Seq when sending writes due to the following reasons: First, it is difficult to coordinate numerous clients simultaneously to get a global Seq; Second, the number of clients can fluctuate at any time. Therefore, we have deployed a Seq dispatcher within the switch (the dispatcher location shown in Fig. 4 to allocate incrementally increasing global Seq values to replace local Seq values in write packets, like Fig. 7(a), ensuring the proper functioning of the stash mechanism.

Observing Fig. 7(a), we notice that the global Seq $\geq$ local Seq. However, in our reliability mechanism, clients set timeout

timers based on the local Seq. If the WRITE_REPLY packet returned by the server after processing maintains the global Seq and is transmitted back to the client, it will result in the deletion of timers and data backups with sequence numbers between the global Seq and the local Seq. This disruption would affect the reliability mechanism of Magneto. Moreover, assuming that after passing through the Seq dispatcher in Fig. 7(a), the write from Client 1 with Seq=3 and the write from Client 0 with Seq=4 share the same key. If an overwrite occurs in the stash, it causes the data with Seq=3 to be overwritten. After the synchronization between the switch and server, the WRITE_REPLY packet with Seq=4 will be sent to Client 0. This will cause the timer for Seq=3 in Client 1 to expire, triggering retransmission and resulting in incorrect data updates. This inconsistency represents a second issue that compromises reliability.

To fix the two aforementioned issues, we have established a lookup table, as shown in Fig. 7(b). Each entry in the table corresponds to a client, identified by (IP:port), where the addition or removal of a client corresponds to the addition or deletion of an entry in the table. In each entry, besides the client, there are two columns: one recording the maximum global Seq obtained by this client, and the other recording the local Seq for this global Seq. This setup aims to establish the mapping between client, Max_seq, and its corresponding local Seq. The switch simultaneously updates this table when writes pass through the Seq dispatcher.

For the first issue, when the WRITE_REPLY packet arrives at the switch, the switch looks up the table based on the destination IP address and port to retrieve the mapping between Max_seq and local Seq. It then replaces the Seq in the packet with the local Seq obtained from the table before forwarding it to the client. This replacement ensures that timers and data backups with Seq values between the global Seq and the local Seq are not cleared.

And for the second issue, in addition to forwarding the WRITE_REPLY packets with the global Seq replaced by the corresponding local Seq, the switch broadcasts WRITE_REPLY packets to all clients whose Max_seq is less than the Seq in the received packet. Before broadcasting, these packets are modified based on the client information in the table, replacing the global Seq with the corresponding local Seq and modifying the destination IP address and port number accordingly.

At this point, even if the write from Client 1 with Seq=3 and the write from Client 0 with Seq=4 share the same key, and an overwritten scenario in the stash results in the data with Seq=3

being overwritten, the WRITE_REPLY packet can still be sent to Client 1. This prevents the timer for Seq=3 in Client 1 from expiring and triggering retransmission.

C. Data Consistency and Fault Tolerance Proof

1) *System Model and Definitions*: The system is modeled as a state transition system $\langle \Sigma, \rightarrow \rangle$, where $\Sigma = (T, S, C, \text{Seq}_{\text{global}})$. Here, T represents the stash state containing key-value pairs with counters, S denotes the server state for persistent storage, $C = \{C_1, C_2, \dots, C_N\}$ represents client backups of unacknowledged writes, and $\text{Seq}_{\text{global}}$ is a monotonically increasing global sequence number. The transition relation $\rightarrow$ captures operations such as write, sync, failure, and recovery. The system maintains three invariants: Stash-Server Consistency ensures that after a sync operation, for all keys $k \in T \cap S$, the values in S and T are identical, i.e., $S(k) = T(k)$. Client Backup Validity guarantees that every client backup $C_i \in C$ is a subset of the writes present in T or pending sync. Total Ordering ensures that for any two writes w_1 and w_2 , if $\text{Seq}_{\text{global}}(w_1) < \text{Seq}_{\text{global}}(w_2)$, then $w_1 \prec w_2$.

2) *Formal Proof of Data Consistency*: Theorem 1 (Strict Consistency) states that for any key k and at any time t , the value of k observed by a client is the latest write present in either T or S . The proof proceeds by induction. Initially, $T = \emptyset$ and S is consistent, satisfying the base case. For the inductive step, assume the invariants hold at time t . When a write operation $w(k, v, \text{Seq})$ is performed, if $k \in T$, the value $T(k)$ is updated, and $\text{Seq}_{\text{global}}$ is incremented by 1; otherwise, T is updated to include the new key-value pair with a sequence number of 1. During a sync operation, when $|T| \geq k$, the contents of T are batched to S , updating S with the key-value pairs from T and removing the corresponding entries from T . A read operation prioritizes the stash T if the key k is present; otherwise, it retrieves the value from S . These operations preserve the invariants, ensuring that clients always observe the latest value from either T or S , thus preventing stale reads.

3) *Formal Proof of Fault Tolerance*: Theorem 2 (Eventual Consistency Post-Failure) asserts that after a switch failure, all unacknowledged writes are eventually recovered through client retransmissions. When a failure occurs, the set of lost writes $F \subseteq T$ is identified. Clients detect timeouts for their respective backups C_i and retransmit the writes with exponential backoff until acknowledged. The switch rebuilds the stash T as $T' = \bigcup_{i=1}^N C_i \cup S$. The liveness property ensures that for each write $w \in F$, after a maximum of R retries, w is delivered to T' . The total recovery time is bounded by $O(R \cdot \max(\text{RTT}))$. Safety is maintained by mapping retransmitted writes using their original local sequence numbers to global sequence numbers via a lookup table, ensuring no duplicates or conflicts arise due to total ordering.

4) *Scalability Analysis With Formal Constraints*: For a system with M servers and N clients, the stash size under a Zipf(α) distribution is $S = O(\frac{\log M}{1-\alpha})$, which is sufficient to cover 99% of hot keys. The system throughput scales linearly with the number of clients, given by $\Theta(N \cdot \lambda)$, where λ is the switch processing rate. In a Spine-Leaf topology with K ToR switches,

each switch i maintains its own stash T_i and global sequence number $\text{Seq}_{\text{global}_i}$. The FPGA memory limitation restricts the maximum stash entries to $|T_{\text{max}}| = \lfloor \frac{B}{\text{avg entry size}} \rfloor \approx 10^4$ entries, where B is the FPGA memory size. For a Fat-Tree topology with L layers, the sync latency is $\Delta_{\text{sync}} = O(L \cdot \text{hop delay})$. Clients connect to d switches, achieving an availability of $P(\text{available}) = 1 - (1 - p)^d$, where p is the switch uptime.

5) *Formal Guarantees Summary*: The system ensures linearizability, meaning all operations appear to execute atomically at some point between invocation and response. Fault tolerance is guaranteed, with all writes eventually becoming part of $S \cup T$ after a failure. Scalability is maintained through linear throughput growth with the number of clients and sublinear stash size growth with the number of servers.

D. The Appropriate Size of Stash.

1) *Definition of Zipf Distribution*: The probability mass function (PMF) of the Zipf distribution is typically defined as:

$$P(k) = \frac{1/k^\alpha}{\sum_{i=1}^{|K|} (1/i^\alpha)} \quad (1)$$

where k is the rank of the key, $|K|$ is the total number of keys, and α is the skew parameter of the Zipf distribution, which controls the skewness of hot keys.

2) *Mathematical Analysis*: The hot key set H can be defined as the top S keys with the highest frequencies according to the Zipf probability distribution. This implies:

$$H = \{k \mid k \in K, P(k) \geq P(|K| - S + 1)\} \quad (2)$$

The cumulative probability $C(S)$ of the hot keys can be represented as the sum of the probabilities of all hot keys. Due to the properties of the Zipf distribution, this can be approximated as:

$$C(S) \approx \sum_{k=1}^S P(k) = \sum_{k=1}^S \frac{1}{k^\alpha |K|^{\alpha-1}} \quad (3)$$

The stash coverage rate $C_{\text{coverage}}(S)$ represents the ratio of the request frequency that the stash can handle to the total request frequency. Considering α , this can be expressed as:

$$C_{\text{coverage}}(S) = \frac{C(S)}{\sum_{k=1}^{|K|} P(k)} \quad (4)$$

As the stash size S increases, the coverage rate theoretically also increases. However, due to the properties of the Zipf distribution, particularly the influence of α , the rate of increase in coverage diminishes. We can analyze this relationship by solving $C_{\text{coverage}}(S)$. The optimal stash size S^* to maximize the coverage rate now considers α :

$$S^* = \arg \max_S C_{\text{coverage}}(S, \alpha) \quad (5)$$

The parameter α directly affects the distribution of hot keys. A larger α implies a higher frequency for a smaller number of hot keys, potentially allowing a smaller stash S to achieve a high coverage rate. As S increases, the coverage rate $C_{\text{coverage}}(S)$ increases, but the rate of increase slows due to the long-tail property of the Zipf distribution. There exists an optimal stash

size S^* , dependent on α and $|K|$, which maximizes coverage given limited resources.

IV. IMPLEMENTATION

We developed a prototype of Magneto as a simplified key-value store to evaluate performance improvements of setting up stash and cache on the overall system. It includes both the hardware component of the in-switch data plane and the software components of the client and server sides. The functionality of system is based on packet interactions between components. Therefore, we designed a standard Magneto packet format, and a client library converts user queries into the standard format before issuing them. We reused the UDP format, ordering the sequence number (Seq), key, value, and other fields as the UDP data payload, to establish connections with the server using sockets.

We built upon the open-source NetFPGA-SUME reference_switch project [25] and integrated hardware primitives into the pipeline to utilize switch-side functionality. Hardware primitives implemented in the in-switch data plane consist of four parts: stash, cache, Seq dispatcher, and parser, all implemented on FPGA, with 6.5 k lines of Verilog code. The stash utilizes TCAM (Ternary Content Addressable Memory) for key matching and RAM (Random Access Memory) for storing variable-length values. An extended bitmap is employed to indicate resource occupancy and count multiple writes for the same key. Meanwhile, by leveraging FPGA parallelism, we detect conditions for triggering sync while buffering new writes. And we set the sync threshold $k = 0.8$ in the prototype, and when the stash's bitmap detects that resource usage exceeds 80%, sync is triggered. At this point, the key-value pairs buffered in the stash are organized as SYN packets in batches and forwarded to servers. The stash's bitmap sets a flag for each memory cell to indicate whether it is occupied. When the flag is set to 1, it means the position is occupied; when set to 0, the position is free and can accommodate new data. After receiving the acknowledgment packet from the server for Max_seq, the bitmap flags corresponding to the synchronized key-value pairs are set to 0. The implementation of the cache is similar to the stash, but it only handles reads. It receives inserts of hot key-values from the server periodically and does not have the synchronization mechanism present in the stash. As for the Seq dispatcher, it maintains a global counter and a lookup table implemented in RAM. After receiving each write, the counter increments by 1, and update the entry with the Max_seq and local seq sent by the same client. The parser separates read and write requests based on the value of the OP field. Specifically, after extracting the relevant fields from the packet, the parser forms a Packet Header Vector (PHV) [22] and then forwards it to either the Seq dispatcher or the Cache for further processing.

The client and server are implemented in Python without using Intel DPDK [26] to optimize I/O performance, as our focus is solely on comparing the overall performance with or without stash. The client can generate read/write queries according to a Zipf distribution at up to 8 MQPS (Million Queries Per Second) with the 10 G NICs (Network Interface Cards) on servers and

translates user queries to Magneto packets. It also runs a sub-thread that keeps a backup for each write sent by the main thread and sets a timeout timer. Then, server confirms the received writes in sequential order based on their Seq and replies with a WRITE_REPLY packet confirming that the write with the maximum Seq has been received by the server.

V. EVALUATION

In this section, we evaluate Magneto and analyze the overall performance under different conditions, including with and without stash, different workload skewness, and varying read/write ratios. The results demonstrate that Magneto provides significant performance improvements on throughput for high-performance key-value stores, and effectively balances load allocated to the storage servers under write-intensive workloads.

A. Experimental Setup

Testbed: Our testbed is consisted of a NetFPGA-SUME, a switch, and two physical machines. Each machine is equipped with a 24-core CPU (Intel Core i9-13900 K), 32 GB memory (Kingston DD5 memory), 1 TB hard disk (Samsung 980pro), and one 10 G NIC (TP-Link). One machine is used as a client to generate key-value queries, while another as the storage server. And NetFPGA-SUME has four 10 G SFP+ interfaces, two 64 MB StrataFlash (PC28F512G18 A), and Two 4 GB DDR3 memory (MT8KTF51264Hz-1G9E1).

Workloads: We normalize the skewness parameter, referred to as *zipf* with its value ranging from 0 to 1. We utilize a mathematic method to generate workloads following the Zipf distribution with different *zipf* values (i.e., 0.9, 0.95, 0.99), which are typical workloads for evaluating key-value stores [27], [28] and are evidenced by real-world deployments [29], [30]. Thus, we primarily employ both skewed and uniform workloads in our experiments to evaluate the performance of key-value stores. This allows us to validate the stash's ability to handle write-intensive workloads and assess overall performance improvement. Keys are randomly selected from a range of 1 to 100,000 and hashed consistently to partition the keyspace. Each key is 16 bytes long, while the value is fixed at 128 bytes, the average size observed in Twitter clusters. Our experiments use a stash with varied sizes and a cache size of 1 k items. Most experiments involve write-intensive workloads, where the proportion of writes exceeds 10%, aligning with Magneto's focus on write-intensive scenarios.

Machine Settings: The switch is able to provide 32 10 Gbps ports. It connects the client and server via optical fiber in a snake way like [7] and routes key-value traffic to NetFPGA-SUME for processing instead of directly forwarding it to the servers. We need multiple servers to evaluate the overall system performance improvement resulting from the stash and other experimental objectives. These include determining the most appropriate stash size and assessing the load balancing effect of various server load under write-intensive workloads. Due to limited available machines, we set up 16 queues in the physical server to simulate 16 clients and 16 storage servers, each handling queries for a key-value partition. We adjust the client's sending rate to match this

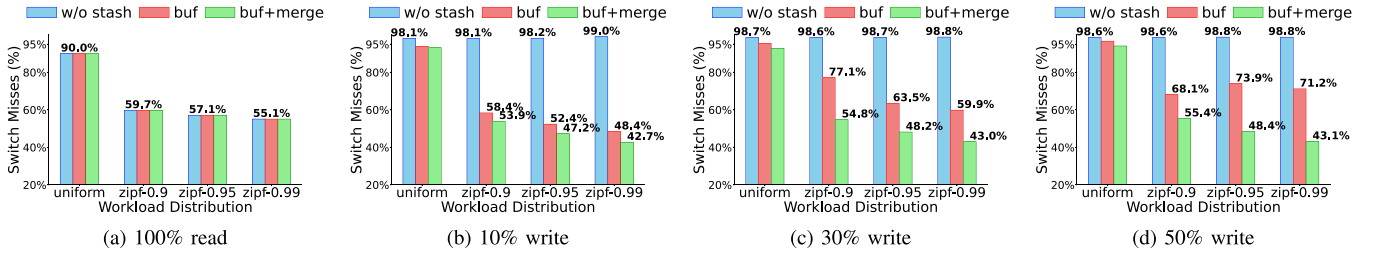


Fig. 8. Switch miss vs. write ratio.

setting and prevent packet loss in queues. However, all queues share a physical NIC in the same physical machine, reducing the throughput of the simulated server to 1/16 of the original throughput. Therefore, the final performance is multiplied by a factor of 16 to obtain a reasonable figure. This setup is justified because our focus lies on relative performance changes across metrics, not absolute values.

B. Microbenchmark Results and Analysis

The impact of skewed writes on in-switch cache: In real production, write queries follow the same distribution as read queries. Therefore, in our experiment setting, write queries share the same skewed set of hot keys as read queries to simulate this feature. Fig. 8 shows that we configured four control groups to test the impact of having a stash or not on switch misses under different distributions, while handling a workload composed of varied write ratios. Switch misses refer to the percentage of requests that cannot be satisfied within the switch, with a higher value indicating less effective in-network processing. When operating without a stash, skewed writes invalidate hot items in the cache, resulting in the benefits of in-switch caching disappearing. Subsequent reads then suffer from cache misses, leading to switch misses exceeding 95% across all distributions. With the introduction of an in-switch stash, cache misses are redirected to the stash rather than being directly sent to the servers. The stash temporarily holds hot writes and can respond to read queries during this period. As a result, under the distributions of zipf-0.9, zipf-0.95, and zipf-0.99, switch misses significantly decrease compared to when the stash is not enabled. Fig. 8 also reveals the reasons why systems like NetCache experience a sharp decline in performance when handling write-intensive workloads.

Additionally, we divided the in-switch stash setup into two groups: one where the stash only buffers write requests and another where the stash both buffers and merges write requests targeting the same key. This division allowed us to assess the impact of buffering and merging writes on reducing switch misses. The results showed that both groups effectively reduced switch misses compared to having no stash at all. However, in the group that only buffer writes, switch misses increased as the proportion of write operations grew. Conversely, the group that both buffer and merge writes maintained its effectiveness, with no significant decline in performance despite the increasing proportion of write operations.

Throughput vs. key-value size: We populate the switch cache with 1 k items and set the stash size as 1 k. One physical

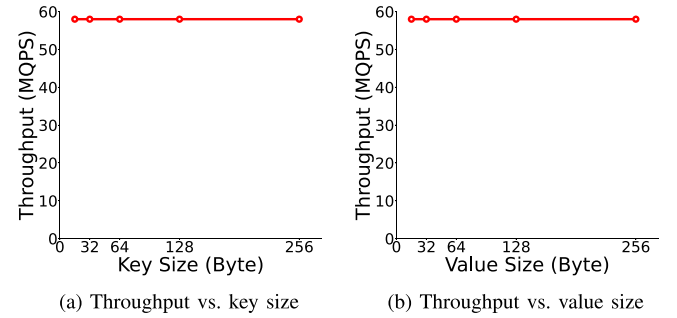


Fig. 9. Switch microbenchmark (Uniform load).

machine acts as the client and sends read and write queries to the switch to measure the maximum throughput. As mentioned before, a query reaches the switch and will be sent to FPGA for processing. Most queries targeting hot items will be processed in the FPGA without involving servers, so the throughput is bottlenecked by the maximum sending rate of the client (3.6 MQPS). Fig. 9(a) shows the system provides 58 MQPS throughput for key sizes up to 256 bytes. Specifically, we have 16 simulated clients (simulated by snake test) sending queries at a rate of 3.6 MQPS, resulting in an overall throughput of 58 MQPS. The hardware primitives we set in the FPGA will process key-value queries in the full pipeline at a frequency of 200 MHz, which means each query just needs 5 ns. The throughput is not affected by the key/value size. Our prototype is implemented using FPGA + switch, which is not limited by resources and programming rules. In theory, it can support larger key sizes by setting a wider team for matching without incurring a latency penalty. As for the value size, its settings are the same as the key's. Fig. 9(b) also shows that the overall throughput is 58 MQPS and is not affected by value size.

The impact of different workloads on sync count: We evaluate the number of synchronizations triggered by the delayed write mechanism under varying read/write ratios and skewness levels. Each workload consists of 1,000,000 queries generated by the clients. Meanwhile, we record the normalized number of queries received by the backend servers under each workload to reflect the load pressure. As shown in the Fig. 10, the count of synchronizations increases with the write ratio, but the backend server load remains relatively stable, consistently within 40% to 60% of the original workload. Moreover, under the same write ratio, more skewed workloads lead to fewer synchronizations and lower backend server load.

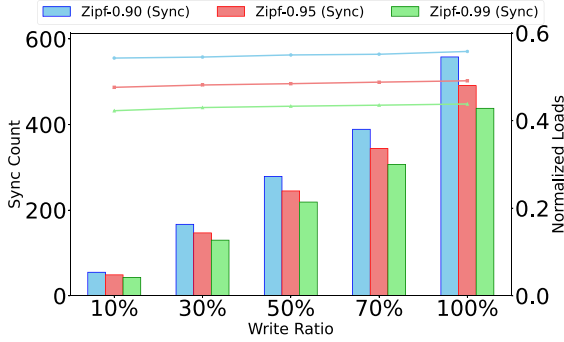


Fig. 10. Sync count and workloads.

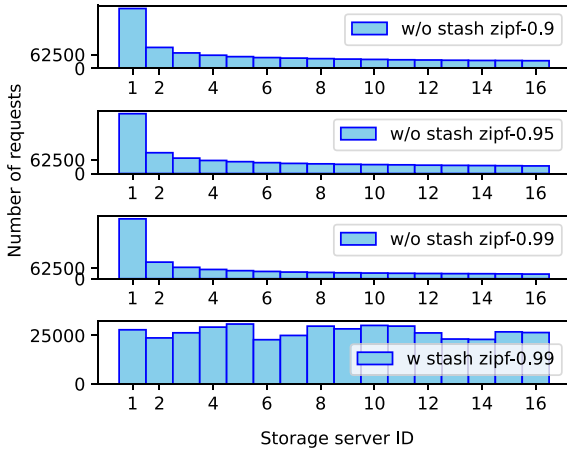


Fig. 11. Load on individual storage servers.

Load sent to individual storage servers: We sent 1,000,000 queries with a 7:3 read-to-write ratio to 16 simulated servers. Under balanced load conditions, each server should handle 62,500 requests within its capability. However, as shown in the Fig. 11, in the first three groups with only read cache, some servers are severely overloaded while others are underloaded. The overloaded servers become the bottleneck, reducing the overall system performance. In contrast, the fourth group with the stash shows that enabling the in-switch stash significantly decreases the number of requests sent to the backend servers, resulting in a more even distribution of load across the servers and avoiding instances of overloading.

C. System Performance

We now evaluate the system performance of a Magneto key-value storage rack under read-intensive workloads (100% read) and write-intensive workloads (70% read + 30% write). This rack contains one switch and 16 servers using server rotation as in [7].

Skewness: Fig. 12(a) shows the system throughput under 0.9, 0.95, and 0.99 skewness parameters with 100% read queries and 1 K hot items in the cache of Magneto. We compare Magneto with None and NetCache under different workloads and show the system throughput provided by them. The NetCache has a steady performance even when the workload is skewed. As for None, it performs poorly when the workload is skewed.

Specifically, under the Zipf-0.95 (0.99) distribution, the None throughput drops down to only 19.77% (17.58%), compared to the throughput under the uniform workloads. Magneto also sets up a cache, thus effectively mitigating the load imbalance and improving the overall throughput.

Meanwhile, we also evaluate the system throughput under different skewness parameters with 70% read and 30% write, namely write-intensive workloads. And write queries follow the same distribution with read respectively. Fig. 12(b) shows that the throughput of NetCache under skewed load drops down to 17.60%, compared to the throughput under uniform workloads. This occurs because write queries invalidate cached items in the in-switch cache when arriving at ToR switch. Therefore, write queries cause cache misses for read queries on the invalidated items instead of benefiting from caching. The system performance degrades to the bottom as the NoCache. While Magneto incorporates a stash module to buffer write queries and subsequently execute delayed writes back to storage servers. In Magneto, missed read queries will check stash subsequently instead of being sent to servers directly. If the stash is hit, it functions as the cache. Therefore, even under Zipf-0.99 workloads, the overall system performance of Magneto reaches 58 MQPS, improved by 8.4x over the NetCache, shown as in Fig. 12(b).

Write ratio: We conducted experiments using write-only queries under different skewed parameters to evaluate the effect of stash on the improvement of the system performance, with a stash size set to 1 K. Fig. 12(c) shows that the overall system performance improves as the skewness of workloads from Zipf-0.9 to Zipf-0.99. At the Zipf-0.99 distribution, the system achieves maximum performance. As mentioned before, fewer items account for the most queries in the more skewed workloads, so more queries would be absorbed in the stash. This result shows that the stash mechanism will effectively benefit skewed write queries.

D. The Impact of the Stash Size

We generated a fixed number of write queries from clients, with keys ranging from 1 to 10,000. Then, we recorded the change in the amount of load received by the storage servers as we varied the size of the stash. This approach aimed to balance both resource utilization and performance, helping us identify the optimal stash size. Fig. 13(a) shows that when the stash size is set to 200, server load sharply decreases under skewed write queries. When the size is increased to 1000, under Zipf-0.99 (Zipf-0.90) distributions, server load decreases to 44% (55%) of the original load. Further increasing the stash size could potentially reduce server load to as low as 10% (not shown in the graph).

We also evaluate the impact of stash size on the system throughput, by continuously sending write queries from clients to saturate the link bandwidth, with keys ranging from 1 to 10,000. While varying the stash size, we recorded the maximum system performance under different skewed parameters as shown in Fig. 13(b). When the stash size is 0, Zipf-0.99 exhibits the lowest system performance due to exacerbated load

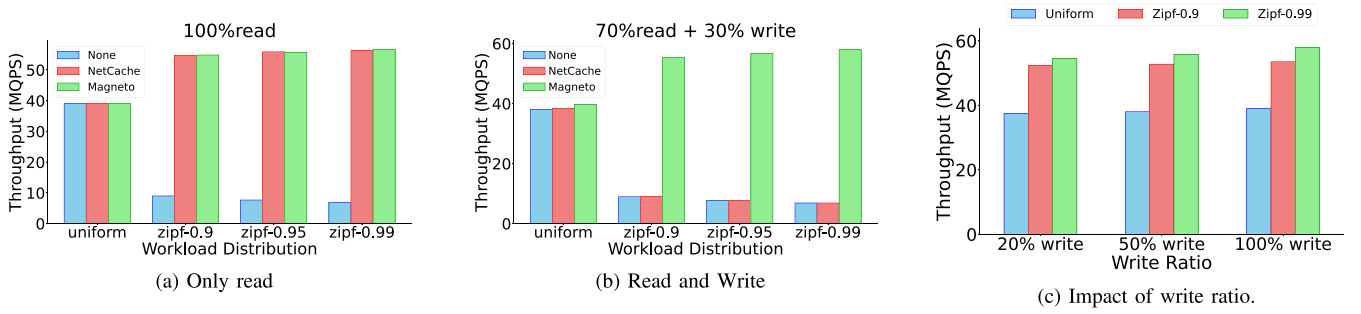


Fig. 12. Throughput vs. skewness and write ratio.

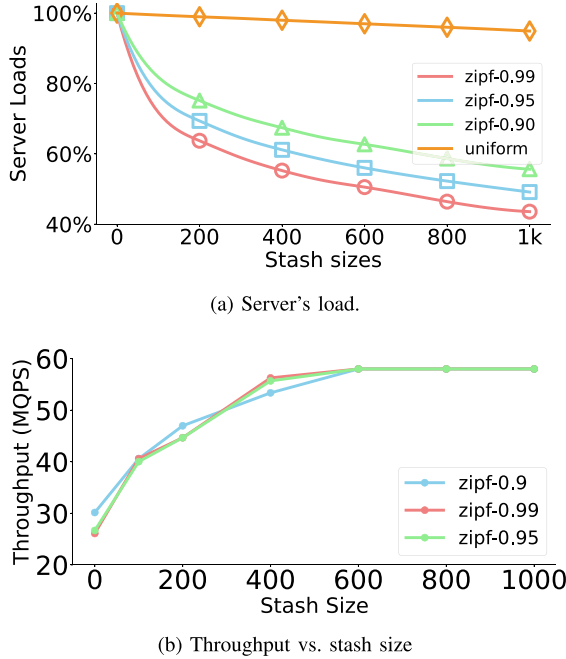


Fig. 13. The impact of the stash size.

imbalance among storage servers. With a stash size of 100, the system performance is significantly improved under three distributions, which indicates that frequent writes are effectively absorbed by the stash. As the stash size increases, Zipf-0.99 achieves the maximum performance first, followed by Zipf-0.95, which aligns with the feature of skewed load. From the stash size reaches 600 onwards, the system performance reaches its maximum for all three distributions, indicating the balanced load among servers.

As mentioned in Section III-A, in-switch stash buffers write queries before meeting the sync conditions. Compared to directly writing to the servers, write queries experience additional delay in the stash. We tested the maximum latency caused by the in-switch stash, as shown in Fig. 14. As the stash size increases, the latency of write queries also increases; the latency under the Zipf-0.99 distribution is greater than that under Zipf-0.9 consistently. It is important to note that this latency represents the time experienced by the first write query buffered in the stash before each sync. Therefore, it is the maximum latency for write queries in the stash. When the stash size is 1000, the system performance reaches its maximum, with 36 microseconds latency under the Zipf-0.99 distribution. Considering the effectiveness

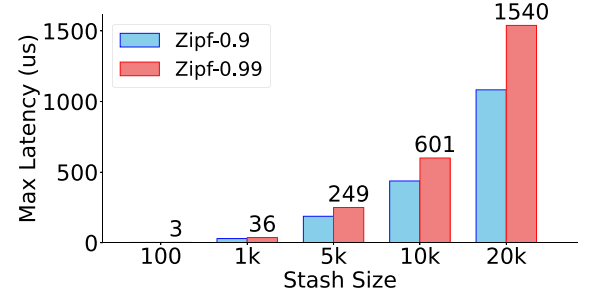


Fig. 14. Latency incurred by stash.

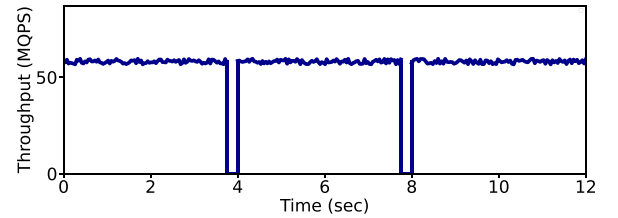


Fig. 15. Throughput changes during switch failure events. (Simulated).

of stash in balancing write-intensive workloads, the latency of 36 microseconds is generally acceptable for most scenarios. The overall latency will be lower when considering the reduced delay in server-side queues due to the setting of the in-switch stash.

E. Fault Tolerant and Resource Usage.

As shown in Fig. 15, if the switch fails, system performance immediately drops to zero and stops receiving new queries. Once the switch resumes working, the system restores performance by receiving queries again after the timed-out retransmitted packets recover the original data from the stash. This assumption is based on the switch quickly recovering from the failure.

Our hardware primitives of the prototype are based on the reference_switch provided by NetFPGA SUME, appended with Cache, Stash, and Seq dispatcher logic implemented in Verilog. As shown in Table I, both Cache and Stash sizes are set to 1000, Magneto utilizes 88,400 LUTs (20.41%), 426.4 BRAMs (29.01%), and 116,794 FFs (13.48%), representing a slight increase compared to the Reference_switch. FPGA has sufficient resources to accommodate the hardware logic required by Magneto.

TABLE I
FPGA RESOURCE USAGE

	LUT	BRAM	FF
Reference_switch	46540 10.74%	289.50 19.69%	79478 9.17%
Magneto	88400 20.41%	426.4 29.01%	116794 13.48%

VI. DISCUSSION

In this section, we will explain the processing flow of read queries in Magneto and discuss the types of hardware suitable for its primitives, as well as the limitations posed by the limited memory of programmable switches on in-network computing.

Read queries in Magneto: Upon receiving key-value traffic, the parser separates read and write queries, then forwards read queries to the Cache, as depicted in Fig. 4. If the Cache hits, the switch crafts a READ_REPLY packet, which is sent back to the client. Otherwise, read queries continue to check if the key-value item exists in the stash. Based on this result, the decision is made either to send it to the server or to construct a READ_REPLY packet to return to the client. Write queries cause massive invalidation of cache entries, which leads to a sharp decline in system performance under write-intensive workloads, especially when they share the same distributions as read queries. Magneto buffers write queries for a while before synchronization. Write queries buffered in the stash have a newer version than the server. During this period, subsequent read queries can retrieve data from the stash instead of flocking to servers. Meanwhile, this approach also avoids fetching outdated data from the server.

Hardware primitives: The in-network processing method depicted in Fig. 4 is decoupled from specific implementation details, allowing flexibility in hardware design. If future switch ASICs have sufficient memory resources, the hardware primitives described in this paper could still be migrated to ASICs for implementation. This design approach enables adaptability and scalability, allowing for the implementation of our method in a variety of hardware environments while retaining its core functionality and benefits. The FPGA's clock frequency, which is at least an order of magnitude slower than ASICs, limits the performance of FPGA-integrated switches. Therefore, our prototype serves only to validate the function of hardware primitives designed in this paper.

Limitations and Future work: While our evaluation demonstrates Magneto's effectiveness under skewed and write-intensive workloads, we acknowledge certain limitations in the current study. First, our experiments primarily focus on synthetic workloads following static Zipf distributions and different read/write ratios. Real-world scenarios may also involve dynamic patterns such as traffic bursts, time-varying locality shifts, and periodic workload fluctuations (e.g., daily peaks in social media platforms). Second, while we validate Magneto's reliability during switch failures, transient network anomalies or cascading failures in multi-rack deployments remain unexplored. Finally, our prototype uses FPGA-integrated switches,

whereas future programmable ASICs may impose stricter resource constraints.

To address these limitations, we plan to extend Magneto's evaluation to more complex scenarios, including (1) replaying real-world application traces (e.g., social media update logs or LLM fine-tuning pipelines) to capture temporal locality and burstiness, (2) integrating adaptive mechanisms to handle dynamic skewness and periodic load patterns, and (3) exploring hybrid hardware deployments combining programmable switches with emerging memory technologies (e.g., CXL-attached memory pools). These enhancements will further validate Magneto's adaptability in production environments while maintaining its core design principles.

VII. RELATED WORK

A. In-Network Caching

SwitchKV [27] directs queries for hot keys to an in-memory cache node using software switches. Its advantages will be diminished if the in-memory storage is adopted by servers. NetCache [7] similarly leverages the highly skewed nature of workloads to build a cache in programmable switches capable of accommodating $O(N \log N)$ hot key-values. The switches directly respond to read requests. However, NetCache is only designed for read-only workloads, as its performance sharply declines when the proportion of write queries exceeds 5%. DistCache [9] provides provable load balancing across multiple racks, but it is also only optimized for read workloads. PKache [31] implements a generic framework that employs limited associativity design and P4 for in-network caching, but it also does not focus on write-intensive workloads. Concordia [32] introduces a cache coherence protocol backed by programmable switches to reduce high coherence overhead. FarReach [6] recognizes the increasing proportion of write queries and implements a write-back caching with a co-designed controller and data plane to tackle write-intensive workloads. However, limited bandwidth between switches and the controller poses a challenge for deployment because the controller may need to communicate with many switches simultaneously.

B. Hardware Acceleration

IncBricks [8] utilizes middleboxes to accommodate more hot key-value items to serve read queries, thereby improving system performance. KV-Direct [33] offloads the functionality of the CPU's key-value processing to programmable NICs, eliminating server CPU intervention in the key-value access path. However, it does not leverage the highly skewed feature of workloads. If combined with the aforementioned in-network caching methods, it could achieve higher system performance. C-4 [34] also targets write-intensive workloads in key-value stores but proposes solutions from the perspective of servers' NICs. It argues that static write partitioning is the fundamental reason for the inefficiency of modern key-value implementations in handling write-intensive workloads. In C-4, read and write queries are handled by independent threads. Additionally, it categorizes keys into dependent or independent classes and assigns them

to different threads for processing. By consolidating multiple writes into a single update, the system enhances processing efficiency.

VIII. CONCLUSION

This paper introduces Magneto, a novel key-value store architecture designed to address load imbalance challenges under write-intensive workloads. By leveraging the skewness of read/write queries, Magneto effectively balances the load among storage servers under write-intensive workloads. This approach leads to significant performance improvements, achieving an 8.4x higher throughput than traditional methods. Additionally, Magneto maintains data consistency and offers fault tolerance, making it a promising solution for scaling key-value stores in demanding service environments.

REFERENCES

- [1] F. Chang et al., "Bigtable: A distributed storage system for structured data," *ACM Trans. Comput. Syst.*, vol. 26, no. 2, pp. 1–26, 2008.
- [2] B. Atikoglu, Y. Xu, E. Frachtenberg, S. Jiang, and M. Paleczny, "Workload analysis of a large-scale key-value store," in *Proc. 12th ACM SIGmetrics/Perform. Joint Int. Conf. Meas. Model. Comput. Syst.*, 2012, pp. 53–64.
- [3] S. Idreos and M. Callaghan, "Key-value storage engines," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2020, pp. 2667–2672.
- [4] Z. Cao, S. Dong, S. Vemuri, and D. H. C. Du, "Characterizing, modeling, and benchmarking {RocksDB} {Key-Value} workloads at facebook," in *Proc. 18th USENIX Conf. File Storage Technol.*, 2020, pp. 209–223.
- [5] J. Yang, Y. Yue, and K. V. Rashmi, "A large-scale analysis of hundreds of in-memory key-value cache clusters at twitter," vol. *ACM Trans. Storage*, vol. 17, no. 3, pp. 1–35, Aug. 2021.
- [6] S. Sheng, H. Puyang, Q. Huang, L. Tang, and P. P. Lee, "{FarReach}: Write-back caching in programmable switches," in *Proc. 2023 USENIX Annu. Tech. Conf.*, 2023, pp. 571–584.
- [7] X. Jin et al., "NetCache: Balancing key-value stores with fast in-network caching," in *Proc. 26th Symp. Operating Syst. Princ.*, 2017, pp. 121–136.
- [8] M. Liu, L. Luo, J. Nelson, L. Ceze, A. Krishnamurthy, and K. Atreya, "Inbricks: Toward in-network computation with an in-network cache," in *Proc. 22nd Int. Conf. ASPLOS*, 2017, pp. 795–809.
- [9] B. Liu et al., "{DistCache}: Provable load balancing for { Large-Scale} storage systems with distributed caching," in *Proc. 17th USENIX Conf. File Storage Technol.*, 2019, pp. 143–157.
- [10] LevelDB. Accessed: 2025. [Online]. Available: <https://github.com/google/leveldb>
- [11] RocksDB. Accessed: 2025. [Online]. Available: <https://github.com/facebook/rocksdb>
- [12] OpenAI. Accessed: 2025. [Online]. Available: <https://chatgpt.com/>
- [13] DeepSeekAI. Accessed: 2025. [Online]. Available: <https://chat.deepseek.com/>
- [14] H. DeCandia et al., "Dynamo: Amazon's highly available key-value store," *ACM SIGOPS Operating Syst. Rev.*, vol. 41, no. 6, pp. 205–220, 2007.
- [15] A. Lakshman and P. Malik, "Cassandra: A decentralized structured storage system," *ACM SIGOPS Operating Syst. Rev.*, vol. 44, no. 2, pp. 35–40, 2010.
- [16] J. Gray, P. Sundaresan, S. Englert, K. Baclawski, and P. J. Weinberger, "Quickly generating billion-record synthetic databases," in *Proc. 1994 ACM SIGMOD*, 1994, pp. 243–252.
- [17] Y. Pan et al., "Sailfish: Accelerating cloud-scale multi-tenant multi-service gateways with programmable switches," in *Proc. ACM SIGCOMM 2021 Conf.*, 2021, pp. 194–206.
- [18] Tofino. Accessed: 2024. [Online]. Available: <https://www.intel.com/content/www/us/en/products/network-io/programmable-ethernet-switch/tofinoseries/tofino.html>
- [19] T. Bosshart et al., "P4: Programming protocol-independent packet processors," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 3, pp. 87–95, 2014.
- [20] G. Bosshart et al., "Forwarding metamorphosis: Fast programmable match-action processing in hardware for SDN," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 43, no. 4, pp. 99–110, 2013.
- [21] F. Chole et al., "DRMT: Disaggregated programmable switching," in *Proc. Conf. ACM Special Int. Group Data Commun.*, 2017, pp. 1–14.
- [22] T. Wang, X. Yang, G. Antichi, A. Sivaraman, and A. Panda, "Isolation mechanisms for {high-speed} { packet-processing} pipelines," in *Proc. 19th USENIX Symp. Netw. Syst. Des. Implementation*, 2022, pp. 1289–1305.
- [23] D. Kim et al., "TEA: Enabling state-intensive network functions on programmable switches," in *Proc. Annu. Conf. ACM SIGCOMM*, 2020, pp. 90–106.
- [24] B. Fan, H. Lim, D. G. Andersen, and M. Kaminsky, "Small cache, big effect: Provable load balancing for randomly partitioned cluster services," in *Proc. 2nd ACM Symp. Cloud Comput.*, 2011, pp. 1–12.
- [25] Reference_switch. Accessed: 2025. [Online]. Available: <https://github.com/NetFPGA/NetFPGA-SUME-public/wiki/NetFPGA-SUME-Reference-Learning-Switch>
- [26] DPDK. Accessed: 2025. [Online]. Available: <https://www.dpdk.org/>
- [27] X. Li, R. Sethi, M. Kaminsky, D. G. Andersen, and M. J. Freedman, "Be fast, cheap and in control with {SwitchKV}," in *Proc. 13th USENIX Symp. Netw. Syst. Des. Implementation*, 2016, pp. 31–44.
- [28] K. V. Rashmi, M. Chowdhury, J. Kosaian, I. Stoica, and K. Ramchandran, "EC-Cache: Load-balanced, low-latency cluster caching with online erasure coding," in *Proc. 12th USENIX Conf. Operating Syst. Des. Implementation*, 2016, pp. 401–417.
- [29] B. Atikoglu, Y. Xu, E. Frachtenberg, S. Jiang, and M. Paleczny, "Workload analysis of a large-scale key-value store," in *Proc. 12th ACM SIGmetrics/Perform. Joint Int. Conf. Meas. Model. Comput. Syst.*, New York, NY, USA, 2012, pp. 53–64.
- [30] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with YCSB," in *Proc. 1st ACM Symp. Cloud Comput.*, New York, NY, USA, 2010, pp. 143–154.
- [31] R. Friedman, O. Goaz, and D. Hovav, "PKache: A generic framework for data plane caching," in *Proc. 38th ACM/SIGAPP Symp. Appl. Comput.*, 2023, pp. 1268–1276.
- [32] Q. Wang, Y. Lu, E. Xu, J. Li, Y. Chen, and J. Shu, "Concordia: Distributed shared memory with {In-Network} cache coherence," in *19th USENIX Conf. File Storage Technol.*, 2021, pp. 277–292.
- [33] R. Li et al., "KV-direct: High-performance in-memory key-value store with programmable NIC," in *Proc. 26th Symp. Operating Syst. Princ.*, 2017, pp. 137–152.
- [34] M. Sutherland, B. Falsafi, and A. Daglis, "Cooperative concurrency control for write-intensive key-value workloads," in *Proc. 28th ACM Int. Conf. ASPLOS*, 2022, pp. 30–46.



Yuanhang Gao received the bachelor's degree in Internet of Things from Zhengzhou University in 2022 and the master's degree in cyberspace security from the National University of Defense Technology (NUDT), in 2025. His research interests include data center networking and computer architecture.



Yingwen Chen received the PhD degree from the National University of Defense Technology (NUDT). He is a professor with the Department of Computer Science, College of Computer, National University of Defense Technology, China. His research interests include network protocols and artificial algorithms for the Internet of Things.



Xiangrui Yang received the PhD degree in computer science and technology from the National University of Defense Technology, Changsha, China, in 2021. He is currently a lecturer with the College of Computer, National University of Defense Technology (NUDT). His research interests include programmable network, data center networking and network security.



Huan Zhou received the PhD degree in computer science from the University of Amsterdam, in 2019. He is currently an associate professor in School of Computer, National University of Defense Technology (NUDT). His research mainly focuses on Cloud computing and network. He is specifically working on Cloud infrastructures seamless programming and control for orchestrating Cloud applications, as well as blockchain enhanced Cloud/Edge service management and next-generation networking technologies.



Ming Xu is currently a professor with the Computer Science Department, College of Computer, the National University of Defense Technology (NUDT), China. His major research interests include mobile computing, wireless network, cloud computing, and network security.



Shihua Tang is working with the National University of Defense Technology (NUDT) as the FPGA engineer in the direction of network hardware devices.